

DS2100

Spring 2026

Python handout: Data Visualization with Matplotlib

We'll use Matplotlib for most of our data viz in this class. Matplotlib is a huge, complex Python library and has the capability to do all kinds of visualizations.

We learn to be good at these visualization tools by reading documentation. Seriously, that's the secret on these libraries. Every professional data scientist googles "how do I...?" for visualization tools, and we are no different in CS2100. Most of the rest of this handout is focused on where to find information and how to interpret it.

Contents of This Handout

[Starting with Matplotlib](#)

[Types of Plots](#)

[Line Chart](#)

[Scatterplot](#)

[Bar Plot](#)

[Histogram](#)

[Customizing Your Plots](#)

Starting with Matplotlib

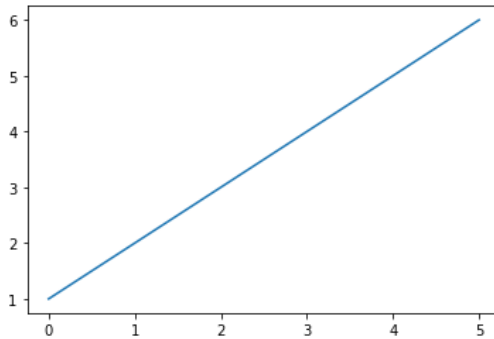
First, you need to import the matplotlib module, which is called `matplotlib.pyplot`. And because “matplotlib.pyplot” is a mouthful, use this import statement at the very top of your .py file:

```
import matplotlib.pyplot as plt
```

This is a shortcut. Now, when you need to refer to matplotlib.pyplot, you just need to say *plt*. If you want to draw a simple line, you can simply provide a list of y-values and matplotlib will assume your x values go 0, 1, 2, Here's an example

```
# Draws a simple line
plt.plot([1, 2, 3, 4, 5, 6, 7])
plt.show()
```

In PyCharm, this plot will pop-up in a separate window, which should be like this:



Plotting a line from one list

You can see in [Matplotlib's documentation](#) that most plot functions, like the one above, return a list of lines representing the plotted data. For the most part, you can simply ignore the return values as we're doing above, but we can also do more sophisticated things if we save and modify those return values.

Types of Plots

If you look up the documentation for different plot types, you'll see what information matplotlib expects and what it will create based on your data. The simple example above is a line chart.

You can modify the colors that are plotted (the default ones are ugly). There is a list of Matplotlib colors here: https://matplotlib.org/stable/gallery/color/named_colors.html

Line Chart

Documentation: https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

We typically use a line chart to show time-series data, and how trends and patterns emerge at even intervals.

The documentation tells you that...

- You create a line chart by calling `plt.plot()`
- You are required to pass in a list of y-values
- You can optionally pass in a list of x-values, which default to [0, 1, 2, ...]
- You can optionally specify colors and markers, such as
 - `plt.plot(x, y, 'g')` # plots a green solid line
 - `plt.plot(x, y, 'g--')` # plots a green dashed line
 - `plt.plot(x, y, 'go')` # plots green circles
 - `plt.plot(x, y, 'ys')` # plots yellow squares

Scatterplot

Documentation: https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.scatter.html

We typically use a scatterplot to show the relationship between two variables.

The documentation tells you (among other things) that...

- You create a scatterplot by calling `plt.scatter()`
- You are required to pass in a list of x-values and a list of y-values
- You can optionally pass in a color or list of colors ([here are matplotlib's color options](#))
- Instead of specific colors, you can also optionally use matplotlib's colormap to plot a range of complementary colors, such as:
 - `plt.scatter(x, y, cmap = "twilight")`
 - `plt.scatter(x, y, cmap = "summer")`
 - `plt.scatter(x, y, cmap = "prism")`

Bar Plot

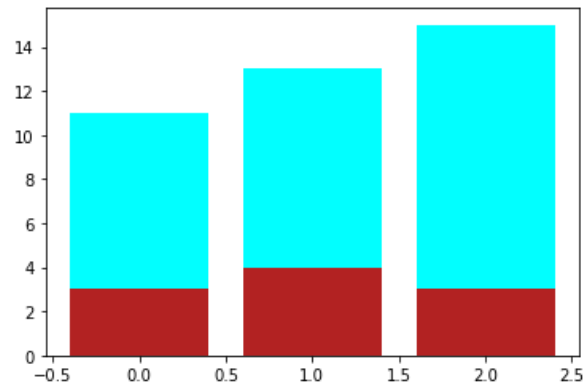
Documentation: https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.bar.html

We typically use a bar plot to show comparisons among different categories. Like line charts, they can also be used to show changes over time.

The documentation tells you (among other things) that...

- You create a bar plot by calling `plt.bar()`

- You are required to pass in a list of x-values, the horizontal positions of the bars and a list of y-values, the heights of the bars
- You can optionally specify the width of each bar
- You can optionally specify the bottom y-value of each bar (this helps with creating stacked bar plots), for example...
 - `plt.bar([0, 1, 2], [3, 4, 3], color = "firebrick")`
 - `plt.bar([0, 1, 2], [8, 9, 12], bottom = [3, 4, 3], color = "aqua")`



Histogram

Documentation: https://matplotlib.org/stable/api/as_gen/matplotlib.pyplot.hist.html

We typically use a histogram to summarize data measured on an interval scale. We put counts of occurrences into bins.

The documentation tells you (among other things) that...

- You create a histogram by calling `plt.hist()`
- You are required to pass in a list values, and it will count occurrences of each distinct value
- You can optionally specify the bins (ranges to group occurrences into, or just a number of bins to group ranges into)

Customizing Your Plots

We expect your DS2500 plots to be clear, easy to understand, and well-labeled. Matplotlib has quite a few ways to customize plots, so here are some suggestions to get started

- **Add axis labels and a title**

Documentation:

- https://matplotlib.org/3.1.1/api/as_gen/matplotlib.pyplot.title.html
- https://matplotlib.org/stable/api/as_gen/matplotlib.pyplot.ylabel.html

- https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.xlabel.html

Examples:

```
plt.xlabel("Label along the x axis")
plt.ylabel("Label along the y axis")
plt.title("Plot Title")
```

- **Add a legend (uses labels)**

Documentation: https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.legend.html

Here's two points with different labels, and then display them using *legend*

```
plt.plot(x1, y1, "s", color = "blue", label = "point one")
plt.plot(x2, y2, "d", color = "orange", label = "point two")
plt.legend()
```

- **Change the range of the axes**

Documentation:

- https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.xlim.html
- https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.ylim.html

Examples

```
plt.xlim(0, 100)
plt.ylim(-100, 100)
```

- **Change the ticks**

Change the tick locations and labels along the x-axis or axis

Documentation:

- https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.xticks.html
- https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.yticks.html

Examples

```
# puts labels a, b,c at positions 1, 2, 3 along the x axis
plt.xticks([1, 2, 3], ["a", "b", "c"])
```

```
# puts labels/xticks rotated sideways
plt.xticks([1, 2, 3], ["a", "b", "c"], rotation = 90)
```

- **Make your chart pop up in VSCode**

Put this command after you're done with one plot; the next time you call `plt.plot()` it will make an entirely new plot beneath the first one.

```
plt.show()
```